Tutoriel pas à pas — lancer, paramétrer et comprendre le projet

> Ce tutoriel part de zéro : installation, premier lancement, modification des paramètres de l'écoulement, puis un parcours de lecture qui fait alterner la théorie (les rapports PDF) et le code (dossier `src/`). Toutes les commandes se tapent dans un terminal ouvert dans le dossier `vortex_project`.

Sommaire

1. Le projet en une page
2. Installer et lancer depuis zéro
3. Modifier les paramètres de l'écoulement
4. Parcours de lecture : la théorie et le code dans l'ordre
5. Dépannage

---

1. Le projet en une page

Le projet étudie les **anneaux tourbillonnaires** (vortex toroïdaux) par trois voies complémentaires :

- la **théorie** : formules exactes ou asymptotiques (vortex de Hill, vitesse de Kelvin et de Saffman, nombre de formation $F^* \approx 4$) ;
- deux **simulateurs** : une méthode lagrangienne 3D en fluide parfait (filaments tourbillonnaires) et un solveur Navier–Stokes visqueux axisymétrique ;
- des **applications** : chambres de combustion à vortex et propulseurs à anneaux pulsés.

Contenu du dossier :

```
vortex_project/
├── README.md            présentation générale, commandes, résultats clés
├── main.py              point d'entrée : lance les calculs et produit toutes les figures
├── requirements.txt     bibliothèques Python nécessaires
├── docs/                la théorie : rapports .md et .pdf, figures, ce tutoriel
├── src/                 le code de calcul (un fichier par grand thème)
│   ├── theory.py                   formules analytiques
│   ├── vortex_particles.py         méthode A : filaments 3D (fluide parfait)
│   ├── solver_2d_axisymmetric.py   méthode B : Navier–Stokes axisymétrique (visqueux)
│   ├── engine_analysis.py          moteurs à vortex
│   ├── visualization.py            figures 2D, bilans, diagrammes
│   └── visualization_3d.py         vues 3D : isosurfaces, tubes, rendu volumique
├── exemples/            quatre scripts courts à modifier pour explorer (section 3)
├── tests/               vérifications automatiques (60 tests)
├── outputs/             résultats produits par main.py (créé au premier lancement)
└── .venv/               environnement Python du projet (créé à l'installation)
```

Le fichier `main.py` pilote tout. Il appelle les modules de `src/`, qui s'appuient sur les formules de `theory.py`, puis il enregistre figures, animations et chiffres clés dans `outputs/`.

---

2. Installer et lancer depuis zéro

2.1 Prérequis

- **Windows 10 ou 11** et **Python 3.10 ou plus récent** (3.12 recommandé). Pour vérifier, tapez `python --version` dans un terminal. Si Python est absent, installez-le depuis python.org en cochant « Add python.exe to PATH ».
- **Facultatif.** `ffmpeg` sert aux animations MP4 ; sans lui, les animations sont écrites en GIF. Chrome ou Edge (Edge est déjà installé sous Windows) sert à régénérer les PDF.
- **Conseillé.** Visual Studio Code, avec l'extension Python, pour lire le code et les rapports.

2.2 Ouvrir un terminal dans le dossier du projet

- **Depuis l'Explorateur de fichiers** : ouvrez le dossier `vortex_project`, faites un clic droit dans une zone vide, puis choisissez « Ouvrir dans le Terminal ». Autre possibilité : tapez `powershell` dans la barre d'adresse, puis Entrée.
- **Depuis VS Code** : *Fichier → Ouvrir le dossier…* → `vortex_project`, puis *Terminal → Nouveau terminal*.

Pour vérifier que vous êtes au bon endroit, la commande `ls` doit lister `main.py`, `src`, `docs`, etc.

2.3 Créer l'environnement virtuel (une seule fois)

L'environnement virtuel `.venv` est une copie de Python propre au projet : les bibliothèques installées ne se mélangent pas avec celles des autres programmes.

```
python -m venv .venv
```

Si le dossier `.venv` existe déjà (c'est le cas sur ce PC), passez directement à l'étape suivante.

2.4 Installer les bibliothèques (une seule fois)

```
.venv\Scripts\python -m pip install -r requirements.txt
```

Cette commande installe numpy (tableaux), scipy (fonctions scientifiques), matplotlib (figures), pillow (GIF) et pytest (tests).

Toutes les commandes du tutoriel appellent explicitement `.venv\Scripts\python` : il n'y a rien à « activer ». Vous pouvez tout de même activer l'environnement avec `.venv\Scripts\Activate.ps1`, puis écrire simplement `python`. Si Windows bloque ce script, voir la section 5.

2.5 Vérifier l'installation avec les tests

```
.venv\Scripts\python -m pytest
```

Le résultat attendu est `60 passed`, en une dizaine de secondes. Chaque test vérifie une propriété physique ou numérique : conservation de l'énergie, vitesse de Saffman, divergence nulle, etc.

2.6 Premier lancement, en mode rapide (≈ 2 min 30)

```
.venv\Scripts\python main.py all --quick --out outputs_rapide
```

La console affiche cinq bandeaux, un par partie : PHASE 1, 2A, 2B, 3, puis VISUALISATIONS AVANCÉES. Suivent l'avancement des calculs, quelques chiffres clés, et enfin `Terminé en … s`. Ouvrez ensuite le dossier `outputs_rapide\figures` : une trentaine d'images PNG y ont été créées.

2.7 Lancement complet (≈ 8 min, ≈ 12 min avec la vidéo)

```
.venv\Scripts\python main.py all
```

Ce lancement reprend les résolutions des rapports et écrit dans `outputs/`. Avec l'option `--docs`, il met aussi à jour les figures utilisées par les rapports (`docs/figures`).

2.8 Lancer une seule partie

Commande	Durée (complet)	Ce qui est calculé	À lire en parallèle
`main.py theory`	quelques secondes	constantes de cœur, invariants de Hill, nombre de formation	théorie § 2–3
`main.py particles`	≈ 2 min	filaments 3D : saute-mouton, collisions frontale et décalée (+ animations)	théorie § 1.5–1.6, § 4.1
`main.py axisym`	≈ 1 min	anneau visqueux, comparaison à Saffman	théorie § 2.3, § 4.2
`main.py engines`	≈ 10 s	chambre à vortex, propulseur pulsé	rapport moteurs
`main.py advanced`	≈ 5 min	saute-mouton visqueux, portrait de phase, tubes, rendus volumiques	théorie § 5

Chaque commande s'écrit `.venv\Scripts\python main.py <partie>`. L'option `--quick` divise les durées par 3 à 5.

2.9 Les options de la ligne de commande

Option	Effet	Exemple
`--quick`	résolutions et durées réduites (essais rapides)	`main.py axisym --quick`
`--out DOSSIER`	dossier de sortie (défaut : `outputs`)	`--out essai_re500`
`--re VALEUR`	nombre de Reynolds $Re_\Gamma = \Gamma/\nu$ de la partie `axisym` (défaut 1000)	`--re 2000`
`--t-end VALEUR`	durée simulée de la partie `axisym` (défaut 20)	`--t-end 30`
`--scenario NOM`	partie `particles` : `leapfrog`, `collision`, `offset` ou `all`	`--scenario collision`
`--animate FORMAT`	animations de `particles` : `mp4`, `gif` ou `none`	`--animate gif`
`--smoke-video`	partie `advanced` : vidéo en rendu volumique (≈ 4 min de plus)	`main.py advanced --smoke-video`
`--docs`	recopie les figures des rapports dans `docs/figures`	`main.py all --docs`

2.10 Où trouver les résultats

- `outputs/figures/*.png` : les figures. Le préfixe indique la partie : `theory_`, `particles_`, `axisym_`, `engine_`, `leapfrog_`, `tubes_`, `smoke_`.
- `outputs/animations/*.mp4` (ou `.gif`) : les animations 3D.
- `outputs/data/axisym_diagnostics.csv` : les séries temporelles de l'anneau visqueux (position, rayon, vitesse simulée et théorique, énergie…). Le fichier s'ouvre dans Excel.
- `outputs/summary.json` : les chiffres clés de chaque partie. Relancer une seule partie ne met à jour que sa rubrique.

2.11 Régénérer les PDF des rapports

Après une modification d'un fichier `.md` du dossier `docs/`, ou après `main.py all --docs` :

```
.venv\Scripts\python docs\build_pdf.py
```

---

3. Modifier les paramètres de l'écoulement

3.1 Grandeurs et unités

Les simulateurs travaillent en **unités adimensionnelles**. Le rayon de l'anneau vaut $R = 1$ et sa circulation $\Gamma = 1$; les temps sont donc exprimés en R^2/Γ et les vitesses en Γ/R. Un anneau typique avance à $U \approx 0{,}2$: il parcourt environ 4 rayons en 20 unités de temps.

Paramètre	Signification	Valeur de référence
R	rayon de l'anneau	1
a (ou σ)	rayon du cœur gaussien	0,1 à 0,3 (anneau mince : $a/R \lesssim 0{,}35$)
Γ	circulation (signe = sens de rotation et de propagation)	1
$Re_\Gamma = \Gamma/\nu$	nombre de Reynolds : plus il est grand, moins la viscosité agit	1000 à 3000
δ	rayon de régularisation des filaments (méthode A), $\delta \approx 0{,}64\,\sigma$	0,1

La partie « moteurs » est en **unités SI** : mètres, m/s, m^3/s.

3.2 Niveau 1 — les options de la ligne de commande

C'est le plus simple : on ne touche pas au code. Pour comparer deux régimes, envoyez chacun dans son propre dossier avec `--out` :

```
.venv\Scripts\python main.py axisym --re 500 --out essai_re500
.venv\Scripts\python main.py axisym --re 3000 --t-end 30 --out essai_re3000
```

À $Re_\Gamma = 500$, le cœur s'épaissit plus vite : l'anneau ralentit davantage et perd plus d'énergie. Comparez les figures `axisym_speed_validation.png` et `axisym_energy_budget.png` des deux dossiers.

```
.venv\Scripts\python main.py particles --scenario collision --animate gif
```

3.3 Niveau 2 — les scripts d'exemple (recommandé pour explorer)

Le dossier `exemples/` contient quatre scripts courts. En tête de chacun, un bloc « PARAMÈTRES À MODIFIER » regroupe les réglages. Le déroulement type :

1. copier le script pour garder l'original intact, par exemple `copy exemples\01_anneau_visqueux.py exemples\mon_essai.py` ;
2. modifier les valeurs du bloc de paramètres ;
3. lancer le script et ouvrir ses figures dans `exemples/resultats/…`.

Exemple 1 — un anneau visqueux (≈ 10 s). Solveur Navier–Stokes ; la console affiche la vitesse simulée comparée à Saffman et à Fukumoto–Moffatt.

```
.venv\Scripts\python exemples\01_anneau_visqueux.py
```

```
R = 1.0          # rayon de l'anneau
A = 0.25         # rayon du cœur gaussien (anneau mince : A/R ≲ 0.35)
GAMMA = 1.0      # circulation (Γ > 0 : l'anneau avance vers les z croissants)
RE = 1000.0      # nombre de Reynolds Re_Γ = Γ/ν (≲ 5000 à cette résolution)
T_END = 8.0      # durée simulée
DR = 0.04        # pas de maillage près de l'anneau : garder A/DR ≥ 6
```

Expériences à tenter :

- faire varier `RE` entre 500 et 3000 et observer le ralentissement et l'énergie perdue ;
- passer `A` à 0,15 avec `DR = 0.025` : un cœur plus fin donne un anneau plus rapide, comme le prévoit $U = \frac{\Gamma}{4\pi R}\left(\ln\frac{8R}{a} - 0{,}558\right)$;
- allonger `T_END` à 20 et vérifier que l'écart avec Saffman + Fukumoto–Moffatt reste de quelques pour cent (voir le tableau de validation, théorie § 4.2).

Exemple 2 — deux anneaux visqueux en saute-mouton (≈ 30 s). La liste `ANNEAUX` décrit chaque anneau : rayon, cœur, circulation, position.

```
.venv\Scripts\python exemples\02_deux_anneaux_navier_stokes.py
```

```
ANNEAUX = [
    {"R": 1.0, "a": 0.15, "Gamma": 1.0, "z0": 2.0},   # anneau arrière
    {"R": 1.0, "a": 0.15, "Gamma": 1.0, "z0": 2.6},   # anneau avant (écart 0.6)
]
RE = 2000.0      # Re_Γ = max|Γ| / ν
T_END = 6.0      # durée simulée
DR = 0.03        # pas de maillage : garder a/DR ≥ 5
```

Avec ces cœurs épais, les anneaux fusionnent dès la première traversée : l'écart entre les cœurs vaut environ 3 fois leur rayon, sous le seuil de fusion (≈ 4). Expériences à tenter :

- affiner les cœurs et reprendre le cas du rapport (`a = 0.10`, `DR = 0.02`, `RE = 3000`, `T_END = 12`, ≈ 2 min de calcul) : on observe alors deux traversées avant la fusion (théorie § 5.4) ;
- écarter les anneaux (`z0` = 2.0 et 2.9) ou donner des rayons différents (1,2 et 0,8) ;
- baisser `RE` à 1000 : la fusion arrive plus tôt.

Exemple 3 — filaments 3D en fluide parfait (≈ 10 s). La variable `SCENARIO` choisit la configuration : `saute_mouton`, `collision`, `collision_decalee` ou `sur_mesure`.

```
.venv\Scripts\python exemples\03_filaments_3d.py
```

```
SCENARIO = "saute_mouton"   # "saute_mouton", "collision", "collision_decalee" ou "sur_mesure"
DELTA = 0.1                 # rayon de régularisation δ
N_NOEUDS = 64               # nœuds par anneau : précision ~1 % si 2πR/N ≤ δ/2
T_END = 10.0                # durée simulée
DT = 0.05                   # pas de temps (≤ 0.05 pour δ = 0.1)
PERTURBATION = 0.02         # saute-mouton : ondulation azimutale qui rend la dynamique 3D
```

Expériences à tenter :

- `PERTURBATION = 0` (anneaux parfaitement circulaires) puis `0.05` : l'ondulation grandit (instabilité 3D) ;
- `SCENARIO = "sur_mesure"` : modifiez les `make_ring(...)` (rayon `R`, circulation `Gamma`, centre `center`, axe `normal`) pour lancer des anneaux inclinés ou de tailles différentes ;
- `N_NOEUDS = 128` : résultat plus précis, calcul environ 4 fois plus long.

Exemple 4 — moteurs à vortex (≈ 5 s, unités SI). Chambre à vortex bidirectionnel et propulseur pulsé.

```
.venv\Scripts\python exemples\04_moteurs.py
```

Pour la chambre, jouez sur `U_INJECTION`, `DEBIT`, `RAYON` et `LONGUEUR` : le paramètre clé $\kappa = Q/(2\pi U a L)$ fixe le nombre de tours et les temps de séjour. Pour le propulseur, modifiez `D_TUYERE` et `U_CROISIERE`, puis observez la carte de rendement.

3.4 Niveau 3 — modifier les réglages de `main.py`

Pour changer les cas qui produisent les figures des rapports, modifiez les valeurs dans les fonctions de `main.py`. Chaque réglage est commenté ; cherchez les lignes suivantes avec Ctrl + F.

Partie	Fonction	Ligne à chercher	Paramètres
`axisym`	`run_axisym`	`ax2d.run_vortex_ring(R=1.0, a=0.2, Gamma=1.0, ...`	rayon, cœur, circulation ; `dr = 0.04 if quick else 0.025` (maillage)
`particles`	`run_particles`	`delta = 0.1`, `sep = 0.6`, `sep = 3.0`, `offset=0.9`	régularisation, écarts des anneaux
`particles`	`run_particles`	`n_nodes, t_end, dt = ...`	nœuds par anneau, durée, pas de temps
`advanced`	`run_advanced`	`sigma0 = 0.10`, `sep = 0.6`, `Re=3000.0`	cœurs, écart, Reynolds du saute-mouton visqueux
`engines`	`run_engines`	`ea.BidirectionalVortexChamber()`	écrire par exemple `ea.BidirectionalVortexChamber(U=60.0, Q=0.08)`
`engines`	`run_engines`	`ea.VortexRingThruster()`, `U_inf = 0.5`	diamètre de tuyère `D`, vitesse du véhicule

Attention : après `main.py all --docs`, les rapports utilisent ces nouvelles figures, alors que leurs textes décrivent encore les cas d'origine.

3.5 Garde-fous numériques

Règle	Pourquoi
Navier–Stokes : au moins 5 mailles par rayon de cœur ($a/\Delta r \geq 5$), 8 pour des résultats de référence	un cœur sous-résolu fausse la vitesse et la dissipation
Navier–Stokes : $Re_\Gamma \lesssim 5000$ avec $\Delta r = 0{,}025$; au-delà, affiner Δr	les gradients deviennent trop raides pour la grille
Navier–Stokes : le pas de temps est choisi automatiquement (critères CFL et visqueux)	rien à régler
Navier–Stokes : le domaine est périodique en z (longueur 10)	un anneau qui sort par le haut rentre par le bas : ce n'est pas une erreur
Filaments : $2\pi R/N \leq \delta/2$ pour une précision de l'ordre du pourcent	discrétisation du noyau de Rosenhead–Moore
Filaments : `DT` ≤ 0,05 pour $\delta = 0{,}1$, et proportionnellement plus petit si δ diminue	stabilité de l'intégration
Filaments : pas de reconnexion ; en dessous de ~2δ entre filaments, la régularisation domine	limite du modèle en fluide parfait
Moteurs : modèle non visqueux, valable pour $\kappa \ll 1$	hypothèses de la solution de Vyas & Majdalani

3.6 Vérifier qu'un résultat est fiable

- **Incompressibilité** : la divergence maximale affichée doit rester autour de 10^{-12}.

- **Bilan d'énergie** : dans la figure du bilan, le résidu $|E_k - E_{th}|/E_0$ doit rester inférieur à environ 10^{-4}.
- **Convergence** : refaites le calcul avec un maillage deux fois plus fin (`DR` divisé par 2) ; les résultats utiles doivent changer de moins de 1 %.
- **Comparaison à la théorie** : pour un anneau isolé, la vitesse doit suivre Saffman + Fukumoto–Moffatt à quelques pour cent près. Pour les filaments, le saute-mouton doit suivre le modèle coaxial.

---

4. Parcours de lecture : la théorie et le code dans l'ordre

Le principe : lire une section des rapports, ouvrir le code correspondant, puis lancer le calcul et regarder la figure. Le code est commenté en français ligne à ligne : chaque appel de fonction y est expliqué.

4.1 Carte des correspondances

Notion	Théorie	Code	Figure	Test
Vortex de Hill	§ 2.1	`theory.hill_*`	`theory_hill_vortex.png`	`test_theory.py`
Vitesse de Kelvin / Saffman	§ 2.2–2.3	`theory.ring_speed`, `theory.saffman_speed`	`axisym_speed_validation.png`	`test_theory.py`
Biot–Savart régularisé	§ 1.5–1.6	`vortex_particles.biot_savart_segments`	`particles_kernel_convergence.png`	`test_biot_savart.py`
Saute-mouton, collisions	§ 2.4	`vortex_particles.leapfrog_scenario`, `theory.CoaxialRingsModel`	`particles_leapfrog_diagnostics.png`	`test_biot_savart.py`
Fonction de courant, Poisson	§ 1.4	`AxisymmetricNavierStokesSolver.solve_stream_function`	`axisym_fields_final.png`	`test_axisymmetric_solver.py`
Projection, pas de temps	§ 4.2	`project`, `step`, `run`	`axisym_vorticity_snapshots.png`	`test_axisymmetric_solver.py`
Critères Q et λ2, dissipation	§ 5.1–5.2	`q_criterion`, `lambda2_criterion`, `dissipation_rate`	`axisym_isosurfaces.png`, `axisym_dissipation_map.png`	`test_advanced_diagnostics.py`
Nombre de formation F*	§ 3	`theory.formation_number`, `VortexRingThruster`	`engine_thruster_formation.png`	`test_engine_analysis.py`

Étape 1 — La vue d'ensemble

Lisez le README et la section 1 de ce tutoriel. Lancez les tests (§ 2.5) et le mode rapide (§ 2.6), puis parcourez les figures sans chercher à tout comprendre.

Étape 2 — Les équations fondamentales

Lisez `docs/theoretical_background.pdf`, sections 0 et 1. Les idées clés :

- la **vorticité** $\boldsymbol{\omega} = \nabla \times \mathbf{u}$ et son équation de transport (étirement, diffusion) ;
- en axisymétrique, seule ω_θ subsiste, et la **fonction de courant de Stokes** ψ donne la vitesse (équation $E^2\psi = -r\omega_\theta$) ;
- la **loi de Biot–Savart** donne la vitesse à partir de la vorticité ; il faut la **régulariser** (Rosenhead–Moore) pour un filament.

Étape 3 — Les modèles analytiques : `src/theory.py`

Lisez la théorie, sections 2 et 3, puis ouvrez `src/theory.py` et lisez les fonctions dans cet ordre :

1. `core_constant`, `ring_speed` : vitesse de Kelvin et de Saffman selon la structure du cœur ;
2. `viscous_core_radius`, `saffman_speed` : anneau visqueux ;
3. `hill_stream_function`, `hill_velocity`, `hill_pressure`, `hill_invariants` : vortex de Hill ;
4. `ring_induced_velocity` : champ exact d'un anneau (intégrales elliptiques) ;
5. `slug_model`, `formation_number` : formation des anneaux ;
6. `CoaxialRingsModel` : modèle réduit de plusieurs anneaux coaxiaux (saute-mouton).

À lancer : `.venv\Scripts\python main.py theory`, puis ouvrir `theory_hill_vortex.png`. À relire : `tests/test_theory.py`.

Étape 4 — Méthode A, filaments 3D : `src/vortex_particles.py`

Lisez la théorie, § 1.5, § 1.6 et § 4.1. Dans le code, suivez la chaîne de calcul :

1. `biot_savart_segments` : vitesse induite par des segments tourbillonnaires (noyau régularisé intégré exactement) ;
2. `Filament`, `ring_nodes`, `make_ring` : représentation d'un anneau par des nœuds ;
3. `VortexFilamentSimulation` : `velocity` (vitesse de tous les nœuds), `step` (Runge–Kutta 4), `remesh` (ajout de nœuds quand les filaments s'étirent), `run` (boucle en temps) ;
4. `leapfrog_scenario`, `head_on_collision_scenario`, `offset_collision_scenario` : configurations prêtes à l'emploi.

À lancer : l'exemple 3, ou `main.py particles --scenario leapfrog --animate gif`. À relire : `tests/test_biot_savart.py`.

Étape 5 — Méthode B, Navier–Stokes axisymétrique : `src/solver_2d_axisymmetric.py`

Lisez la théorie, § 1.3, § 1.4 et § 4.2. Dans le code, suivez l'ordre d'exécution :

1. `AxisymmetricGrid` (dont `stretched`) : grille décalée de type MAC, étirée en r, périodique en z ;
2. `RadialSpectralSolver` : résolution exacte de l'équation de Poisson (FFT en z, décomposition spectrale en r) ;
3. `AxisymmetricNavierStokesSolver` : `solve_stream_function` et `set_vorticity` (champ initial à partir de ω_θ), `init_lamb_oseen_ring`, `rhs` (convection et diffusion), `project` (projection de Chorin : impose la divergence nulle), `step` (Runge–Kutta 4 projeté), `diagnostics` (position, rayon, énergie, dissipation), `run` (boucle en temps) ;
4. `run_vortex_ring` : simulation complète et comparaison à la théorie.

À lancer : l'exemple 1, ou `main.py axisym --quick`. À relire : `tests/test_axisymmetric_solver.py`.

Étape 6 — Diagnostics et visualisations avancés

Lisez la théorie, § 5. Dans le code :

- fin de `solver_2d_axisymmetric.py` : `q_criterion`, `lambda2_criterion`, `dissipation_rate`, `enstrophy_density`, `track_rings` (suivi de plusieurs anneaux), `run_coaxial_rings` ;
- `src/visualization.py` : `plot_energy_enstrophy_budget`, `plot_dissipation_map`, `plot_hovmoller`, `plot_phase_diagram` ;
- `src/visualization_3d.py` : `revolve_field` et `isosurface_of_revolution` (3D par révolution), `tube_mesh` (tubes de vorticité), `volume_render` (rendu volumique).

À lancer : l'exemple 2, ou `main.py advanced --quick`. À relire : `tests/test_advanced_diagnostics.py`.

Étape 7 — Les applications : `src/engine_analysis.py`

Lisez `docs/vortex_engines_analysis.pdf`. Dans le code :

- `BidirectionalVortexChamber` : trajectoires, temps de séjour, pression, gouttes ;
- `cold_wall_heat_flux` : effet « paroi froide » ;
- `reactor_conversion` : conversion chimique ;
- `VortexRingThruster` : formation des anneaux, impulsion, rendement propulsif.

À lancer : l'exemple 4, ou `main.py engines`. À relire : `tests/test_engine_analysis.py`.

Étape 8 — Le chef d'orchestre : `main.py`

Relisez `main.py` de haut en bas :

- `main` lit les options, puis appelle `run_theory`, `run_particles`, `run_axisym`, `run_engines` et `run_advanced` ;
- chaque fonction `run_*` fixe les paramètres de sa partie, lance les calculs, enregistre les figures et renvoie les chiffres de `summary.json` ;
- la liste `DOC_FIGURES` indique les figures recopiées dans les rapports par `--docs`.

Étape 9 — Les tests, fiches de vérification

Chaque test énonce une propriété attendue : c'est souvent la façon la plus courte de comprendre ce qu'une fonction doit faire. Pour afficher le nom de chaque test, ou n'exécuter que ceux qui contiennent un mot :

```
.venv\Scripts\python -m pytest -v
.venv\Scripts\python -m pytest -k hill -v
```

Conseils pour lire le code dans VS Code

- Choisissez l'interpréteur du projet : Ctrl + Maj + P, puis « Python : Sélectionner l'interpréteur » → `.venv`.
- F12 va à la définition d'une fonction ; Maj + F12 liste ses utilisations ; Ctrl + Maj + O affiche le plan du fichier.
- Pour suivre un calcul pas à pas : ouvrez un script d'exemple, placez un point d'arrêt (F9) sur une ligne, puis appuyez sur F5. Si VS Code le demande, choisissez le débogueur Python, puis « Fichier Python ». Le programme s'arrête sur la ligne marquée ; F10 avance d'une ligne, F11 entre dans la fonction appelée. Le panneau « Variables » montre les tableaux et leurs valeurs.
- Les rapports `.md` s'affichent avec leurs formules par Ctrl + Maj + V.

---

5. Dépannage

Problème	Solution
`python` n'est pas reconnu, ou ouvre le Microsoft Store	installer Python depuis python.org en cochant « Add python.exe to PATH », ou utiliser `py` à la place de `python`
`Activate.ps1` est bloqué (« l'exécution de scripts est désactivée »)	ne pas activer : utiliser `.venv\Scripts\python …` ; ou autoriser une fois `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`ModuleNotFoundError: No module named 'src'` (ou `numpy`)	lancer les commandes depuis le dossier `vortex_project`, avec `.venv\Scripts\python`
Pas de fichier MP4, seulement des GIF	`ffmpeg` n'est pas installé : les GIF sont le repli automatique ; ou utiliser `--animate gif`
Le calcul est trop long	ajouter `--quick`, lancer une seule partie, ou réduire `T_END` dans les exemples
Valeurs `nan` ou simulation qui diverge	maillage trop grossier pour le cœur ou Reynolds trop élevé : réduire `DR` ou `RE` (voir § 3.5)
`build_pdf.py` ne trouve pas le navigateur	indiquer le chemin de Chrome ou d'Edge dans la variable d'environnement `CHROME_PATH`